

Getting Started

Executable = machine code (0/1) the CPU runs.
Source (.c) must be **compiled** to an executable.
OS mediates CPU, memory, devices (**Linux** in this course).

Compile & Run (Linux)

```
1 $ gcc file1.c -o run // compile
2 $ ./run // execute (note ./)
```

- Multiple files: `gcc a.c b.c -o app`. *Headers* declare function prototypes; implementations are linked in. `#include <stdio.h>` exposes `printf` & friends.

Karel the Robot

World + Wrapper

```
1 #include <karel.h>
2 int main() {
3     karel_setup("settings/settings00.json");
4     // ... your Karel code here ...
5     turn_off(); // end & goal check
6 }
```

Core Actions

- `move()`, `turn_left()`, `turn_right()`
- `take_item()`, `put_item()`, `turn_off()`

Core Queries

- Walls: `wall_in_front()`, `wall_to_left()`, `wall_to_right()`
- Items/bag: `item_present()`, `bag_empty()`
- Heading: `facing_north()`, `facing_east()`, `facing_south()`, `facing_west()`

Control Patterns

```
1 // IF / ELSE
2 if (item_present()) { take_item(); } else { move(); }
3
4 // Logical ops: && (and), || (or), ! (not)
5 if (item_present() && facing_east()) { take_item(); }
6
7 // Loops: while / repeat(n)
8 while (!wall_in_front()) { move(); }
9 repeat(3) { move(); turn_right(); }
10
11 // Define your own action (function)
12 void turn_until_north() {
13     while (!facing_north()) { turn_left(); }
14 }
```

C Integer Variables

Declare / Assign / Init

```
1 int moves; // declaration
2 moves = 5; // assignment
3 int steps = 10; // initialization
4 int a=1, b=2; // multi-init
```

Arithmetic & Shorthand

```
1 a = b + 3 * 2; // *, / before +, -
2 a += 1; b *= 2; // +=, -=, *=, /=, %=
3 ++a; b--; // pre/post inc/dec
```

Precedence (excerpt)

(), $x++/x--$ → **unary** ($+x, -x, -, !, *, \&$) → $*, /, \%$ → $+, -$ → $<, >, \leq, \geq$ → $==, !=$ → $\&\&$ → $—$ → assignment.

I/O (stdio)

```
1 int x; scanf("%d", &x); // & for input
2 printf("x=%d\n", x); // %d for int
```

`scanf` returns number of successfully read items (useful for validation).

Other Data Types

Integer Families

Basic sizes (typical gcc): `char`=1B, `short`=2B, `int`=4B, `long`=8B, `long long`=8B.
Signed vs Unsigned: add signed/unsigned keyword. Overflow silently wraps.

Conversion Characters

Type	printf/scanf
char/short/int	%d (%u unsigned)
long	%ld (%lu)
long long	%lld (%llu)
float/double	%f %e %g; double also %lf etc.
char as char	%c (digit as number: %d)

Fixed-Width (stdint.h)

```
1 #include <stdint.h>
2 int16_t s; uint32_t u;
```

Characters & ASCII (char)

```
1 char c = 'A'; // also: char c = 65;
2 printf("%c %d\n", c, c); // A 65
```

Floating Point

- `float` (4B, single), `double` (8B, double). No % for modulo on floats.
- Print with `%f %e %g` (and `%lf` for double in `scanf/printf`).

Pointers (int *)

Concepts

- **Address** of a variable: where it lives in memory.
- **Pointer type** holds an *address*. Declare with `*`:
`int *p;`
- **& (address-of)** gives address; *** (dereference)** follows address to the value.

Core Example

```
1 int a=4, b; int *p;
2 p = &a;      // p points to a
3 b = *p + 1;  // b = 5
4 *p = b + 3;  // a = 8 (writes via pointer)
```

Watch the *: in `int *p` it's a declaration; in `*p` it's dereference; between values it's multiplication. Context disambiguates.

scanf needs &

```
1 int x,y, n = scanf("%d%d", &x, &y);
2 // n is 2 if both read ok
```

Pointer Sizes & Types

- On 64-bit systems pointers are typically 8 bytes (all pointer *sizes* equal), but the *types* matter for dereference/operations.

Functions & Flow

Define & Call

```
1 int add(int x,int y){ return x+y; }
2 int main(void){ return add(2,3); }
```

Forward Declarations

```
1 int add(int,int);    // declaration (prototype)
2 int main(){ return add(2,3); }
3 int add(int x,int y){ return x+y; }
```

Jump Statements

- **break** exits innermost loop/switch.
- **continue** skips to next loop iteration.
- **return** exits function (optionally with value).

Memory Organization

Process memory (RAM) split into: **code** (machine instructions), **static data** (globals/statics), **heap** (dynamic alloc), **stack** (function frames, locals). Addresses are byte-indexed; on 64-bit machines addresses are 8-byte values.

Mental Model Tips

- **Stack** grows/shrinks with function calls/returns; locals live here.
- **Pointer diagrams:** draw an arrow from the pointer to the variable it points to; dereference means “follow the arrow”.

Linux CLI Mini-Ref

<code>pwd</code>	show working directory
<code>ls -a</code>	list files (incl. hidden)
<code>cd, mkdir, rmdir</code>	move/create/remove dirs
<code>cp / mv / rm</code>	copy/move/remove files
<code>./ / ../ ~</code>	current / parent / home